

UCS310 – Database Management Systems

2025-2026

BloodLink

An Integrated DBMS for Seamless Blood Bank Management



B.Tech 2nd Year Computer Engineering
Thapar Institute of Engineering and Technology

Shashwat Tewari (1024031065)

Avik Khetarpal (1024031066)

Humza Ali Choudhri (1024031070)

Submitted to: Dr. Chinu Dhir

WEBSITE DEMONSTRATION

BloodLink

Dashboard

Donors

Inventory

Requests

Admin Forms

Dashboard

Smart Blood Bank Management System

Saving Lives Through Better Data Management and Instant Tracking

Total Donors4

Available Units4

Eligible Donors3

Emergency Requests0

Recent Hospital Requests

Refresh

REQUEST ID	HOSPITAL NAME	BLOOD GROUP	QTY	TYPE	DATE	STATUS
#REQ-1	Apollo Hospital	A+	1 Units	Normal	29/04/2026	Pending

Dashboard of our website

BloodLink

Admin Forms

Dashboard

Donors

Inventory

Requests

Admin Forms

Admin Forms

Register New Donor

Full Name

Age

Gender

Blood Group

Phone Number

Email Address

Register Donor

Record Blood Donation

Donor ID

Blood Group

Donation Date

Record Donation

Admin Issue Blood (Stored Procedure)

Admin Forms of our website

INTRODUCTION

2.1 Application Domain

Our project, **Bloodlink**, is a digital platform for managing a **Blood Bank**. At its core, it is an inventory and record-keeping system that connects three different groups: the people who donate blood, the bank that stores it, and the hospitals that need it urgently. Instead of just being a list of names, the system handles "living data"—it tracks things that change constantly, like how many units of O+ blood we have left, which blood bags are about to expire, and which donors are actually fit to donate today.

2.2 Motivation for selecting the problem

We chose this project because blood management is literally a matter of life and death, yet many blood banks still rely on outdated methods. During our research, we realized two big issues: **wastage** and **timing**. Blood isn't like other inventory; it expires in about 35–42 days. If the staff doesn't know which bag is the oldest, they might use a fresh one and let the older one go to waste.

Also, checking if a donor is eligible is usually done by asking them, which can lead to mistakes. We wanted to build something that uses "logic" to solve these problems—like a system that automatically blocks a donation if the person donated too recently. It's a perfect way to apply what we've learned in DBMS to a real-world crisis.

2.3 Importance of DBMS vs File Based System

Using a folder of Excel sheets or a physical register for a blood bank is honestly dangerous. Moving to a DBMS like Oracle or MySQL changes everything:

- **No "Double Booking"**: In a file system, two people could try to assign the same blood bag to two different hospitals at the same time. A DBMS uses **Concurrency Control** to make sure that doesn't happen.
- **Automatic Rules**: We can't trust a human to always check every date. With a DBMS, we can set **constraints** so the system physically won't let you enter a blood type that doesn't exist or a date that is in the future.
- **Faster Searching**: If a hospital calls for "B negative" blood during an accident, searching through a paper file takes minutes. A **SQL Query** takes milliseconds.
- **Data Safety**: Files can be deleted or edited by anyone. A DBMS allows us to set permissions so that only the admin can change the stock levels, while others can only view them.

PROBLEM STATEMENT

3.1 Addressing The Real World Problem

Blood banks face several major challenges in managing blood donation and distribution efficiently.

- **Matching Problem:**
Hospitals require specific blood groups, and strict compatibility rules must be followed to ensure safe transfusions.
- **Expiry Problem:**
Blood is a perishable resource and typically expires within approximately 42 days. Without proper tracking, this can lead to significant wastage of valuable blood units.
- **Safety Problem:**
Donors must maintain a minimum recovery period of 56 days between consecutive donations to protect their health.

3.2 Limitations Of The Existing/Manual System

Manual and traditional systems are generally slow, prone to human errors, and unable to provide timely support during emergency situations. Therefore, there is a strong need for an automated and intelligent system.

3.3 Solving The Problem Using Databases

BloodLink offers a database-driven solution that ensures donor safety, efficient blood inventory management, and fast hospital request fulfillment through automated medical compliance and real-time tracking.

OBJECTIVES OF THE PROJECT

The main objectives of this project are as follows:

- To design an Entity–Relationship (ER) model representing donors, blood units, and hospital requests.
- To convert the ER model into appropriate relational database tables.
- To normalize the database structure up to Third Normal Form (3NF) or Boyce–Codd Normal Form (BCNF) to eliminate redundancy.
- To implement SQL constraints to maintain medical compliance and ensure data integrity.
- To develop database triggers to enforce donor recovery rules and prevent unsafe donations.
- To implement stored procedures for automated hospital request processing and inventory updates.
- To write efficient SQL queries for identifying compatible blood units and supporting emergency matching operations.

SCOPE OF THE PROJECT

The scope of the *BloodLink – Smart Blood Bank Management System* is limited to the design and implementation of a centralized database system that manages blood donors, blood inventory, and hospital blood requests.

5.1 Functional Boundaries Of The System

The system supports:

- Storage and management of donor details and donation history
- Enforcement of donor eligibility rules (minimum 56-day gap) using triggers
- Tracking blood inventory with expiry and availability status
- Compatibility-based blood matching using SQL queries and views
- Processing normal and emergency hospital requests
- Automatic inventory updates with transaction control

The system does not handle medical testing, physical blood collection, transportation, or external system integration.

5.2 Types Of User Involved

- **Blood Donors:** Individuals who register and donate blood.
- **Blood Bank Staff (Admin):** Responsible for managing blood inventory.
- **Hospitals:** Authorized entities that place blood requests.

5.3 Modules Covered

Donor Management Module

- Stores donor personal, medical details and tracks donation history
- Ensures a mandatory 56-day recovery period using database triggers

Blood Inventory Management Module

- Tracks each blood bag with donation and expiry dates
- Monitors availability, expiration status and generates critical stock alerts

Hospital Request Management Module

- Handles normal and emergency blood requests
- Allocates compatible blood unit and updates inventory status automatically

PROPOSED SYSTEM DESCRIPTION

The proposed BloodLink system is a database-driven blood bank management solution designed to automate donor safety, inventory tracking, and hospital request fulfillment using SQL and PL/SQL.

6.1 System Workflow

Donor Registration & Donation

- Donors register their details in the system
- Upon donation, a blood bag entry is created in the Blood_Inventory table
- A database trigger verifies that the donor has completed the 56-day recovery period

Blood Inventory Processing

- Each blood unit is assigned an expiry date (42 days from donation)
- Inventory status is updated automatically
- SQL views identify low-stock and soon-to-expire blood units

Hospital Request Handling

- Hospitals submit blood requests specifying blood group and priority
- A stored procedure checks inventory for compatible blood types
- Blood units closest to expiry are allocated first to minimize wastage

Data Integrity & Automation

- Primary and foreign key constraints ensure consistency
- Transactions (COMMIT/ROLLBACK) prevent partial updates
- Automated procedures reduce human error and response time

6.2 Key Features & System Improvements

- Automated donor eligibility validation
- Perishable inventory management
- Compatibility-based blood matching
- Emergency request prioritization
- Secure and consistent data handling

DATABASE DESIGN

7.1 Identified Entities

1. **Donor**
2. **Hospital**
3. **BloodUnit**
4. **Donation**
5. **Request**
6. **BloodCompatibility**

7.2 Attributes

Donor

- donor_id (PK)
- name
- age
- gender
- blood_group
- phone
- email
- last_donation_date
- next_eligible_date

Hospital

- hospital_id (PK)
- hospital_name
- location
- phone

BloodUnit

- unit_id (PK)
- blood_group
- donation_date
- expiry_date
- status

Donation

- donation_id (PK)
- donor_id (FK)
- unit_id (FK)

Request

- request_id (PK)
- hospital_id (FK)
- blood_group
- quantity

- request_type (Normal / Emergency)
- request_date
- status

BloodCompatibility

- donor_group (PK)
- receiver_group (PK)

7.3 Relationships

Relationship	Type	Explanation
Donor → Donation	1 : N	One donor can donate multiple times over their lifetime
Donation → BloodUnit	1 : 1	Each donation creates exactly one blood bag in inventory
Hospital → Request	1 : N	A hospital can place multiple requests over time
BloodCompatibility	Reference Table	Defines valid donor-receiver blood group pairings

7.4 Constraints

- Donor cannot donate if current date < next_eligible_date
- BloodUnit expires after 42 days
- Donor must wait 56 days between donations
- Blood issued must follow compatibility rules
(Handled via triggers + procedures)

Relational Schema

The Entity–Relationship diagram is converted into the following relational schema. Each entity is represented as a relation (table). Primary keys uniquely identify records, while foreign keys establish relationships among tables.

Donor

Donor(**donor_id**, name, age, gender, blood_group, phone, email, last_donation_date, next_eligible_date)

Primary Key: donor_id

Hospital

Hospital(**hospital_id**, hospital_name, location, phone)

Primary Key: hospital_id

BloodUnit

BloodUnit(**unit_id**, blood_group, donation_date, expiry_date, status)

Primary Key: unit_id

Donation

Donation(**donation_id**, donor_id, unit_id)

Primary Key: donation_id

Foreign Keys:

donor_id references Donor(donor_id)

unit_id references BloodUnit(unit_id)

Request

Request(**request_id**, hospital_id, blood_group, quantity, request_type, request_date, status)

Primary Key: request_id

Foreign Key:

hospital_id references Hospital(hospital_id)

BloodCompatibility

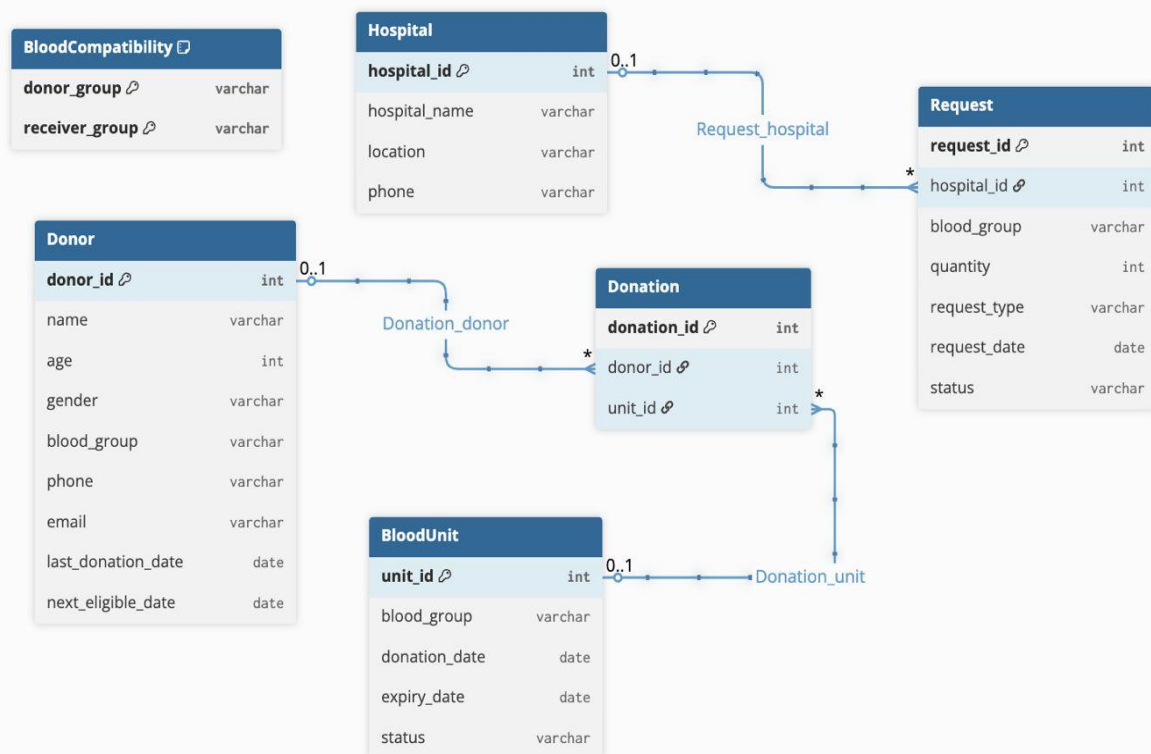
BloodCompatibility(**donor_group**, **receiver_group**)

Composite Primary Key: (donor_group, receiver_group)

Table Relationships

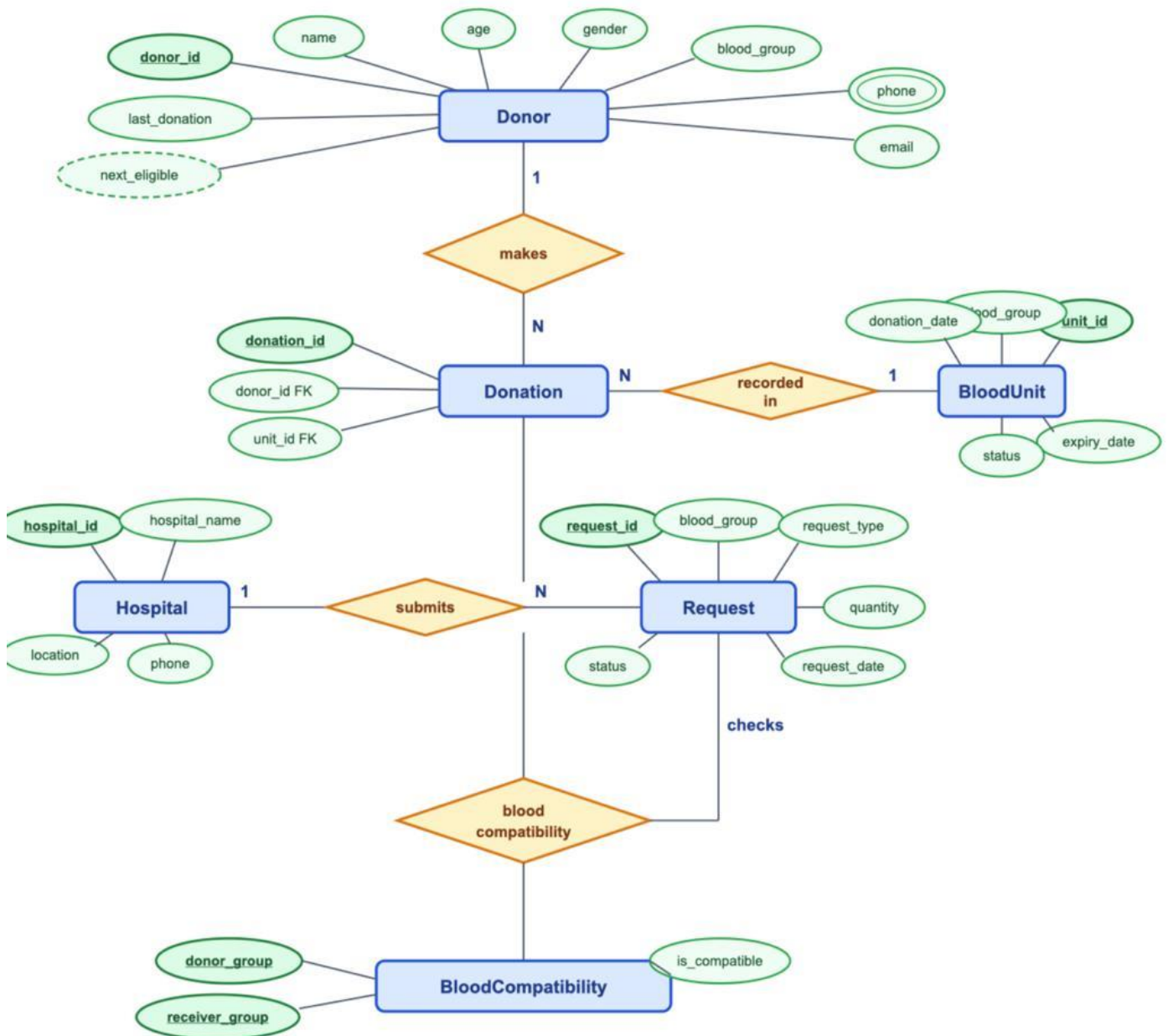
- One Donor can make multiple Donations (1:N relationship).
- Each Donation generates exactly one BloodUnit (1:1 relationship).
- One Hospital can place multiple Requests (1:N relationship).
- BloodCompatibility defines valid donor–receiver blood group mappings using a composite primary key.

Foreign key constraints are enforced to maintain referential integrity between related tables.



Visual Representation of Relational Schema

ER Diagram



Legend — Chen ER Notation (UCS310: Database Management Systems)



Cardinality labels (1, N) beside connecting lines · Underlined = Primary Key · Double oval = Multi-valued · Dashed = Derived

Blood Bank ER Diagram — UCS310: Database Management Systems

Normalization

Normalization is the process of organizing data in a database to reduce redundancy and improve data integrity. The Blood Bank Management System database has been normalized up to **Third Normal Form (3NF)**.

8.1 Identification of Functional Dependencies

DONOR

$\text{donor_id} \rightarrow \text{name, age, gender, blood_group, phone, email, last_donation_date, next_eligible_date}$

HOSPITAL

$\text{hospital_id} \rightarrow \text{hospital_name, location, phone}$

BLOODUNIT

$\text{unit_id} \rightarrow \text{blood_group, donation_date, expiry_date, status}$

BLOODCOMPATIBILITY

(donor_group, receiver_group) is a composite primary key defining valid donor–receiver mappings.

DONATION

$\text{donation_id} \rightarrow \text{donor_id, unit_id}$

REQUEST

$\text{request_id} \rightarrow \text{hospital_id, blood_group, quantity, request_type, request_date, status}$

8.2 First Normal Form (1NF)

A relation is in First Normal Form if:

- All attributes contain atomic values
- No repeating groups or multi-valued attributes exist

All tables satisfy 1NF since each field stores single values only.

8.3 Second Normal Form (2NF)

A relation is in Second Normal Form if:

- It is already in 1NF
- All non-key attributes are fully functionally dependent on the primary key

Since each table uses a single primary key and all attributes depend entirely on that key, no partial dependencies exist.

8.4 Third Normal Form (3NF)

A relation is in Third Normal Form if:

- It is already in 2NF
- There are no transitive dependencies
- Non-key attributes depend only on the primary key

All tables satisfy this condition because:

- Each non-key attribute depends directly on the primary key
- No non-key attribute depends on another non-key attribute

8.5 Final Normalized Tables (3NF)

DONOR

(donor_id, name, age, gender, blood_group, phone, email, last_donation_date, next_eligible_date)

HOSPITAL

(hospital_id, hospital_name, location, phone)

BLOODUNIT

(unit_id, blood_group, donation_date, expiry_date, status)

BLOODCOMPATIBILITY

(donor_group, receiver_group)

DONATION

(donation_id, donor_id, unit_id)

REQUEST

(request_id, hospital_id, blood_group, quantity, request_type, request_date, status)

<u>Table</u>	<u>Attributes</u>
DONOR	donor_id (PK), name, age, gender, blood_group, phone, email, last_donation_date, next_eligible_date
HOSPITAL	hospital_id (PK), hospital_name, location, phone
BLOODUNIT	unit_id (PK), blood_group, donation_date, expiry_date, status
DONATION	donation_id (PK), donor_id (FK), unit_id (FK)
REQUEST	Request_id, hospital_id, blood_group, quantity, request_type, status

Transaction Management & Concurrency

Transaction Management:

The system ensures atomicity during request fulfillment using:

- COMMIT
- ROLLBACK
- SAVEPOINT

Example:

SAVEPOINT before_update;

UPDATE BloodUnit SET status='Issued' WHERE unit_id=101;

UPDATE Request SET status='Done' WHERE request_id=1;

COMMIT;

If any error occurs:

ROLLBACK TO before_update;

ACID Properties:

Property	How BloodLink Implements It
Atomicity	COMMIT and ROLLBACK ensure both updates happen together or not at all
Consistency	Constraints (PK, FK, CHECK) ensure only valid data enters the system
Isolation	Oracle's row-level locking prevents two simultaneous requests from claiming the same blood unit
Durability	Once committed, the data survives system restarts — it's written to disk

Concurrency Control:

Oracle ensures concurrency control using:

- Row-level locking
- Transaction isolation
- Internal locking mechanisms
- **SELECT ... FOR UPDATE** can be used to lock rows during blood allocation.

This prevents double issuing of blood units.

Tools & Technologies Used:

Component	Technology	Reason for Choice
Database	MySQL	Supports PL/SQL natively; triggers and stored procedures work cleanly
Backend	Node.js	Good at handling concurrent requests; the team knew it already
Web Framework	Express.js	Lightweight; lets us set up API routes quickly without a lot of boilerplate
Frontend	HTML5, CSS3, JavaScript	Simple forms for donor registration and admin dashboard; Bootstrap for layout
DB Connector	mysql2	Bridge between Node.js and the database — runs our SQL queries and returns results
Dev Tools	VS Code, MySQL Workbench	Writing and testing code; managing the schema visually

Expected Outcomes

The objective is to deliver a robust, backend-driven database system. By the end of this project, we expect to achieve the following:

- **Normalized Table Structure:** A final database design that satisfies **3NF (Third Normal Form)**, ensuring that there is no data redundancy and that the system remains easy to update without anomalies.
- **Backend Automation via Triggers:** Implementation of **PL/SQL Triggers** to act as automated safety guards. For instance, a trigger will be designed to automatically block a hospital request if the specific blood unit has exceeded its expiration date.
- **Transaction Reliability (ACID Compliance):** Successful implementation of **COMMIT and ROLLBACK** operations within our logic. This ensures that if blood is issued, the inventory is reduced *only if* the request is fully logged, preventing any "ghost" inventory or data mismatches.
- **Modular Logic via Stored Procedures:** We will develop a suite of **Stored Procedures** (such as **Process_Donation** or **Issue_Blood**) that allow users to perform complex operations by passing simple parameters, rather than writing manual SQL queries.
- **Enforced Data Consistency:** Strict application of **Primary Keys, Foreign Keys, and Check Constraints** to ensure that "garbage data"—such as negative stock levels or invalid blood groups—cannot be entered into the system.
- **Simplified Reporting via Views:** The creation of **SQL Views** to provide instant insights, such as a "Low Stock Alert" view that automatically flags blood types falling below a critical threshold.

Conclusion

The BloodLink system successfully demonstrates the design and implementation of a database-driven Blood Bank Management System. The project integrates ER modeling, relational schema design, normalization up to Third Normal Form (3NF), SQL implementation, and PL/SQL components including stored procedures and triggers.

Transaction management using COMMIT and ROLLBACK ensures ACID compliance and data consistency during blood issuance and hospital request processing.

The system effectively handles donor eligibility, blood inventory tracking, compatibility validation, and hospital request fulfillment while maintaining data integrity and minimizing redundancy.

Overall, the project reflects practical application of Database Management System concepts in solving a real-world healthcare problem.